HEALTH REMINDER Design Document

CIS-043-32075 Team 8 Version: v1.1 – April 06, 2026

Date of Submission: Apr 20, 2026

Prepared by: Team 8 **Course:** CIS-043-32075

1. Introduction

Managing healthcare can be challenging, especially for patients who need to keep track of multiple appointments, doctors, and medications. This application is designed to simplify that process by organizing essential health information.

With this app, patients can monitor appointments, maintain a record of their healthcare providers, and track medications with reminders to ensure timely intake.

2. Functional Requirements

2.1 Account Login Allows users to sign up, register, and log into their accounts.

2.2 Medication Management Enables users to create, maintain, edit and delete medication schedules, including dosage amounts. These schedules are integrated with the calendar and reminder system.

2.3 Calendar and Reminders Displays reminders for today's appointments and medications. Enables users to create, edit, and delete appointments.

2.4 Doctors Management Allows users to create, maintain, edit and delete detailed information about their healthcare providers.

2.5 Share Information Enables users to send a summary of their health information (appointments, medications, doctors) via email.

2.6 Application Navigation Flow (See Figure 1 – Proposed Use Case Flow Diagram on the following page or as an appendix image.)

3. Non-Functional Requirements

- **Security**: Passwords will be hashed (never stored in plain text).
- **Performance**: All screens load in < 30 seconds; reminders fire reliably.

- **Usability**: Intuitive tabbed interface with keyboard navigation support.
- **Reliability**: Offline-first desktop application with graceful error handling.
- **Platform**: Windows via JavaFX.

4. Data Model

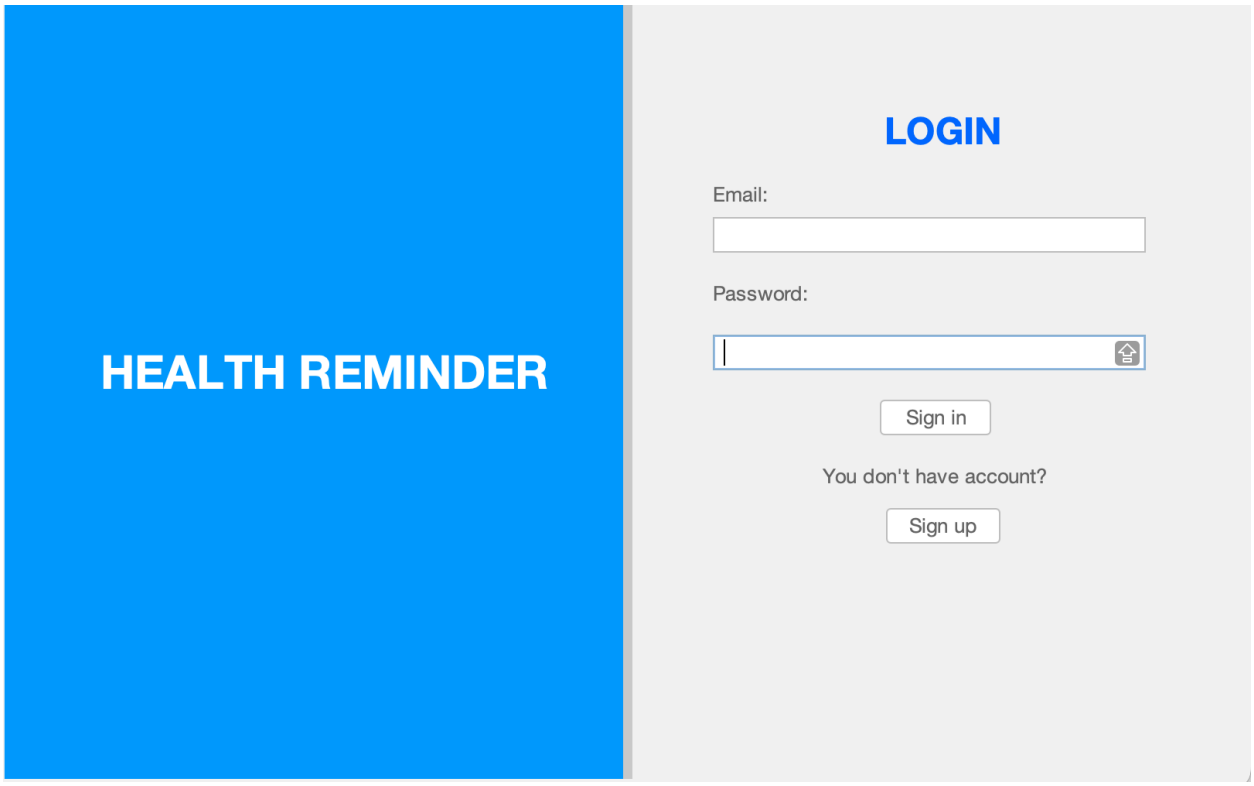
Proposed core entities (to be finalized in UML Class Diagram):

- **User** (id, fullName, email, username, passwordHash)
- **Medication** (userId, name, dosage, frequency, startDate, endDate, notes)
- **Appointment** (userId, dateTime, doctorId, notes)
- **Doctor** (userId, name, specialty, phone, email, address)

5. GUI and UML Design

<u>User Registration & Login</u>

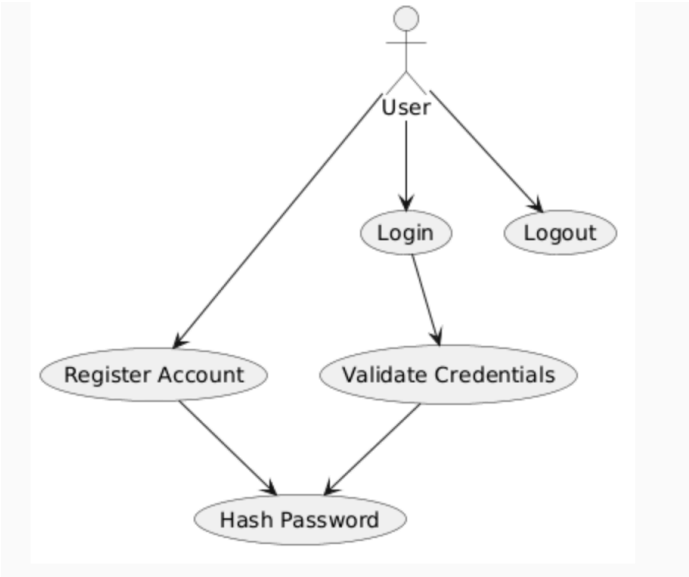
GUI



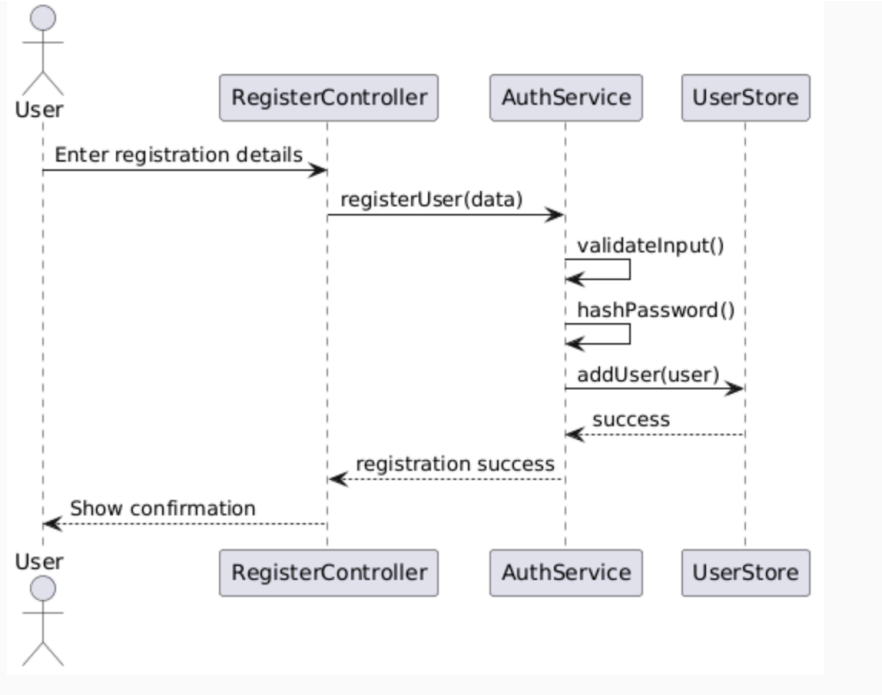
UML Diagrams

All diagrams are suitable for inclusion in a formal **software design document**.

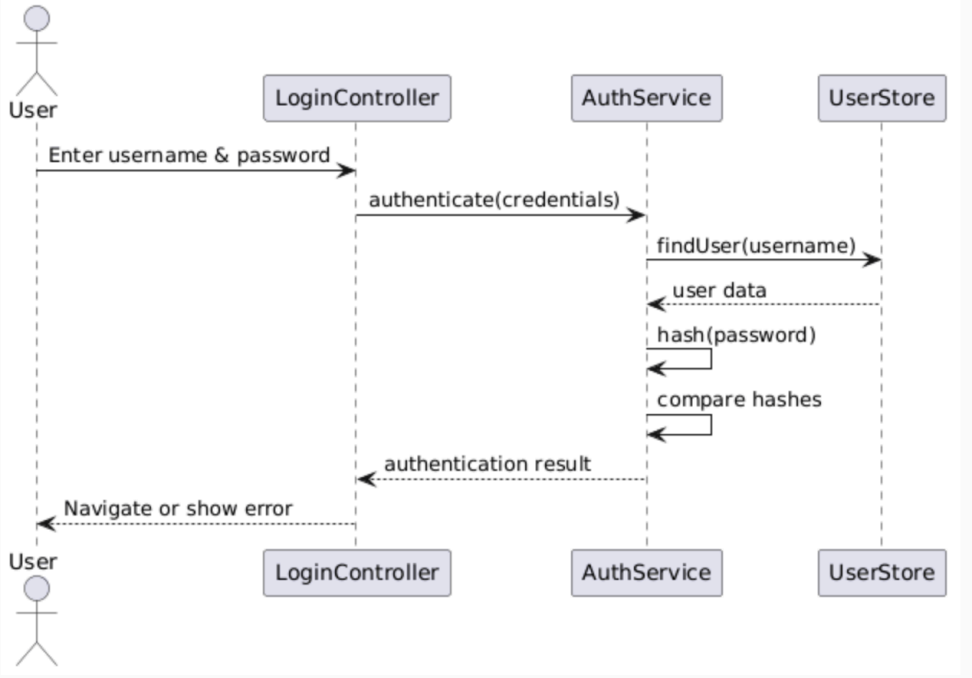
5.1 Use Case Diagram – Registration & Login



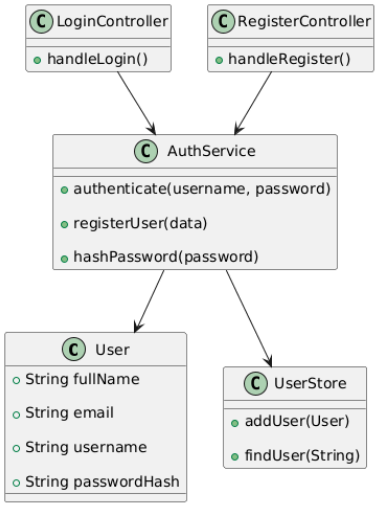
5.2 Sequence Diagram – User Registration (Hardcoded Storage)



5.3 Sequence Diagram – User Login (Hardcoded Storage)

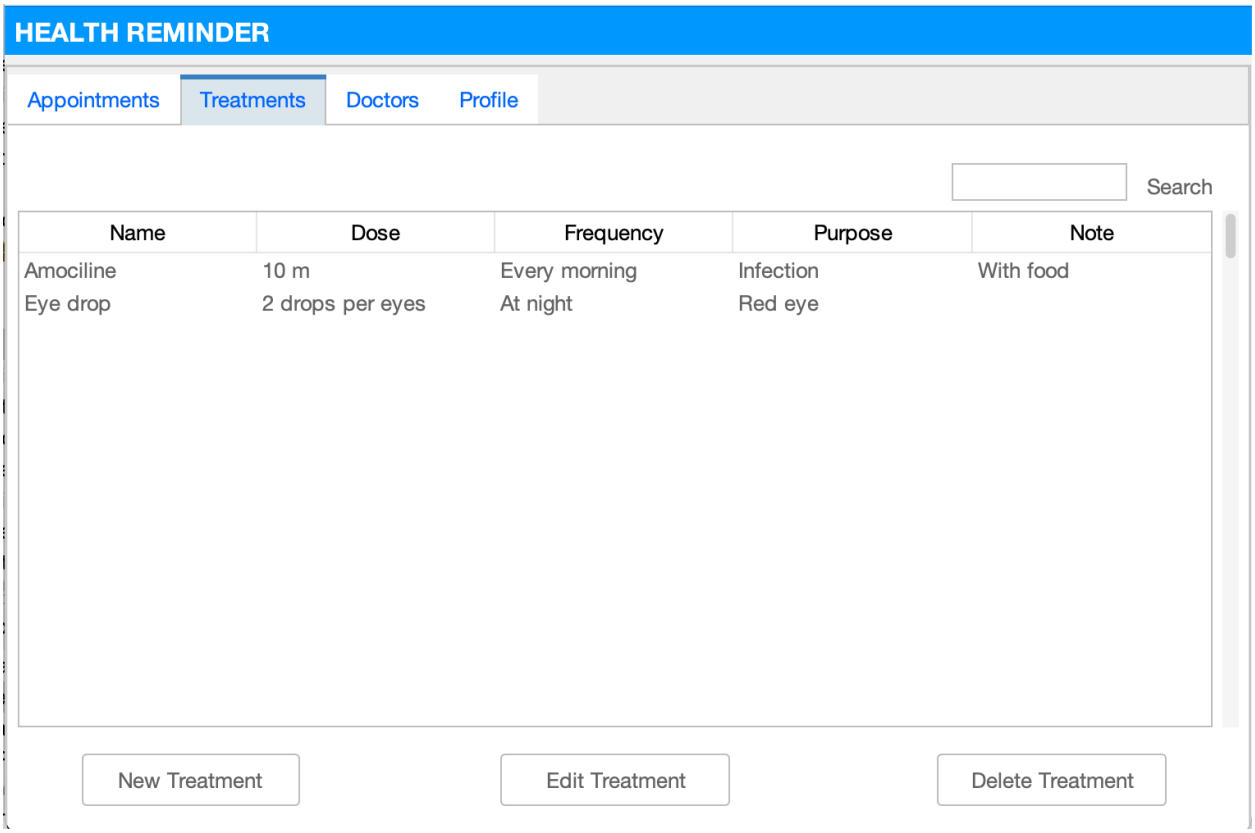


2.4 Class Diagram – Authentication Module (Demo)

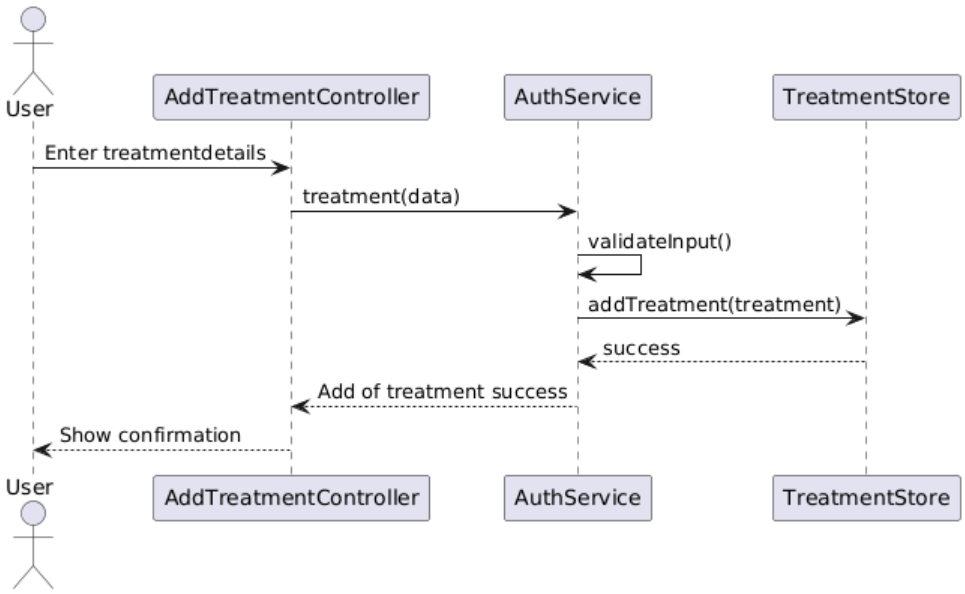


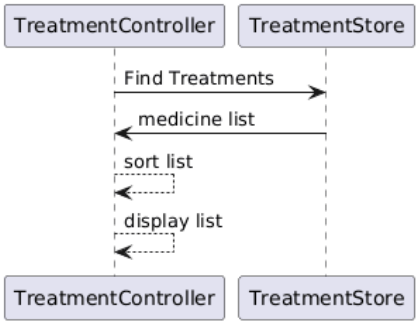
Treatment and Medicine

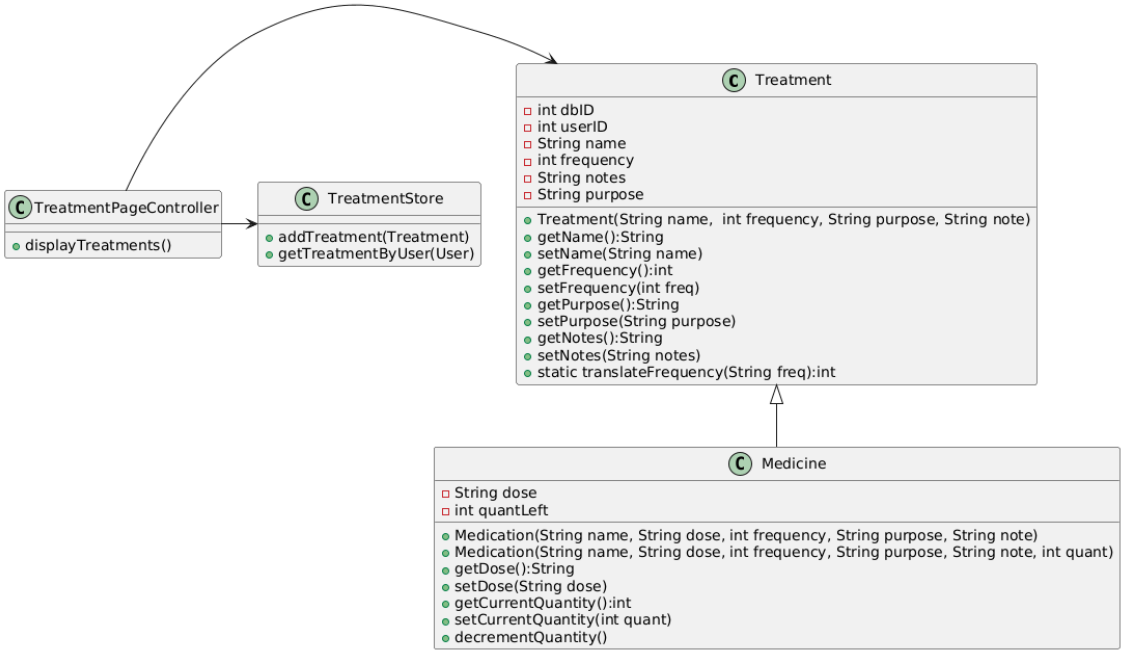
GUI



UML

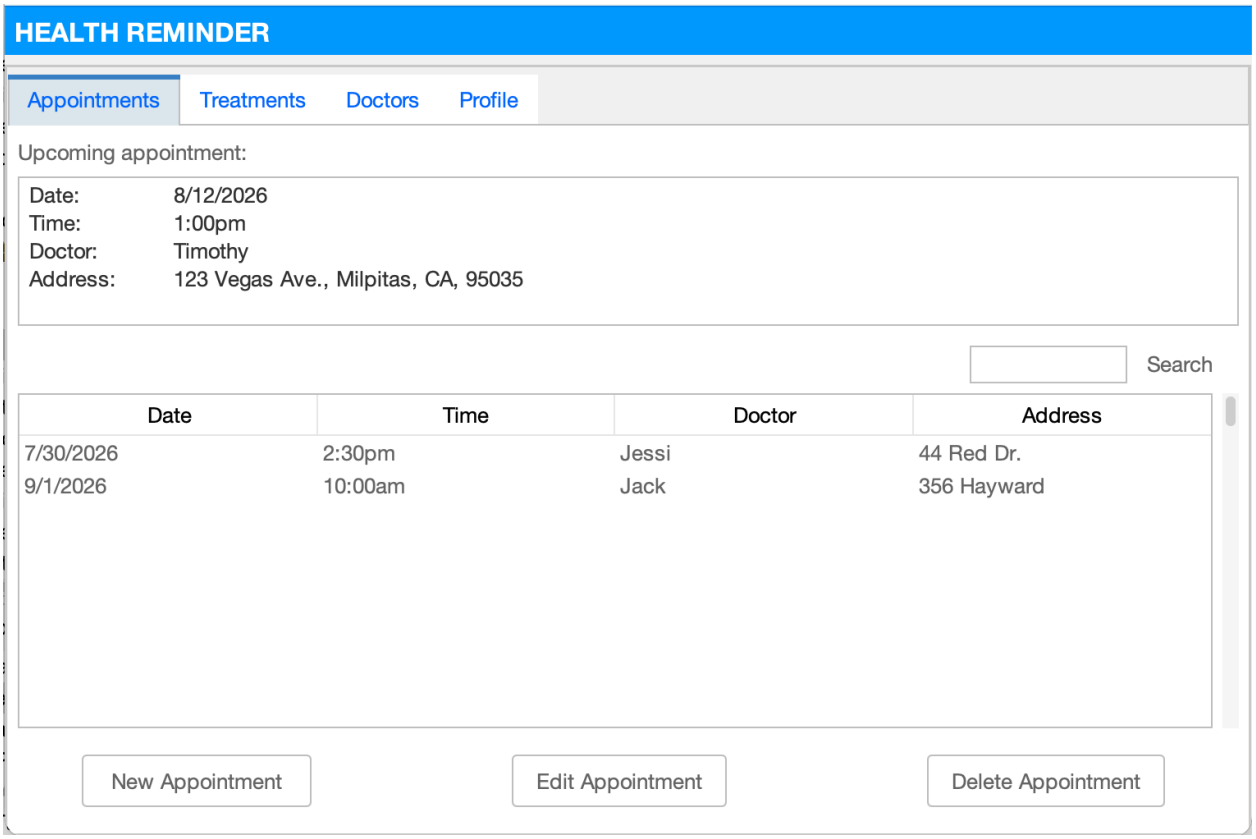



TreatmentController
TreatmentStore
Find Treatments
medicine list
sort list
display list
TreatmentController
TreatmentStore

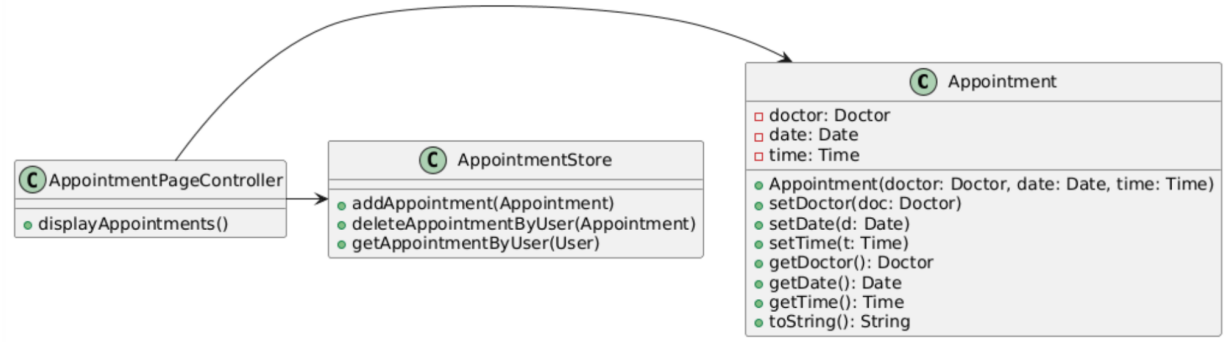

TreatmentPageController
displayTreatments()
TreatmentStore
addTreatment(Treatment)
getTreatmentByUser(User)
Treatment
int dbID
int userID
String name
int frequency
String notes
String purpose
Treatment(String name, int frequency, String purpose, String note)
getName():String
setName(String name)
getFrequency():int
setFrequency(int freq)
getPurpose():String
setPurpose(String purpose)
getNotes():String
setNotes(String notes)
static translateFrequency(String freq):int
Medicine
String dose
int quantLeft
Medication(String name, String dose, int frequency, String purpose, String note)
Medication(String name, String dose, int frequency, String purpose, String note, int quant)
getDose():String
setDose(String dose)
getCurrentQuantity():int
setCurrentQuantity(int quant)
decrementQuantity()

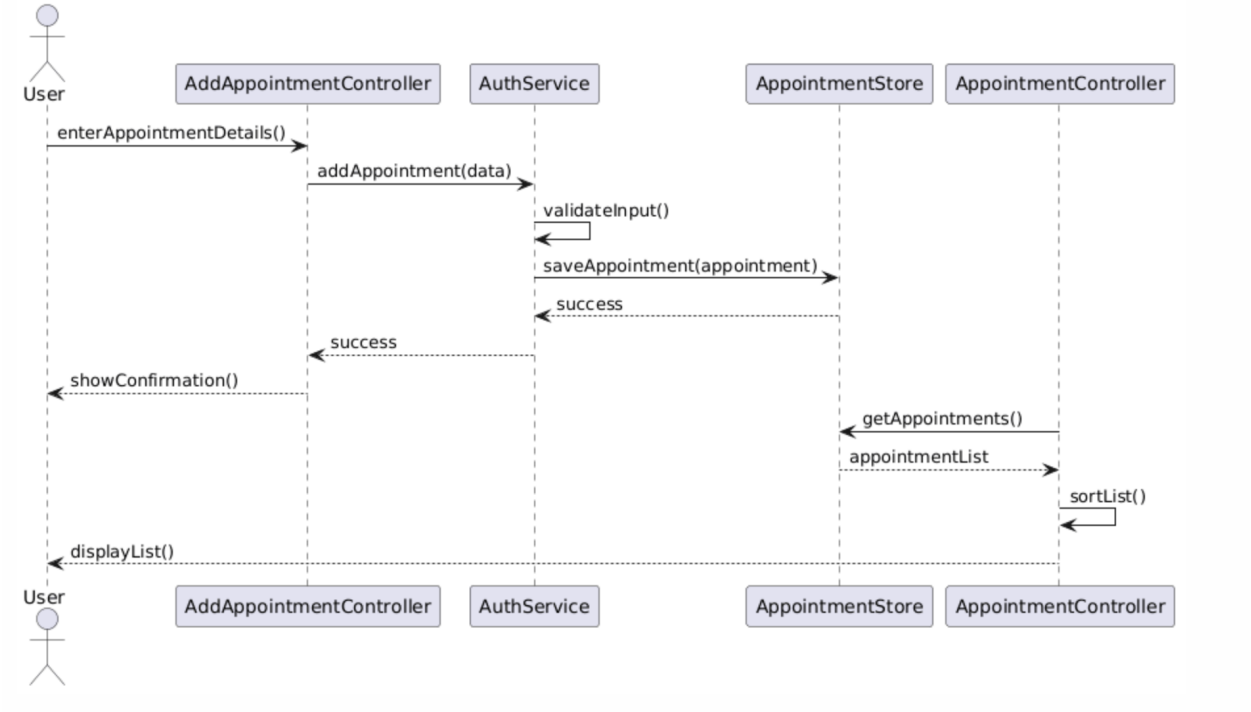
Appointments

GUI



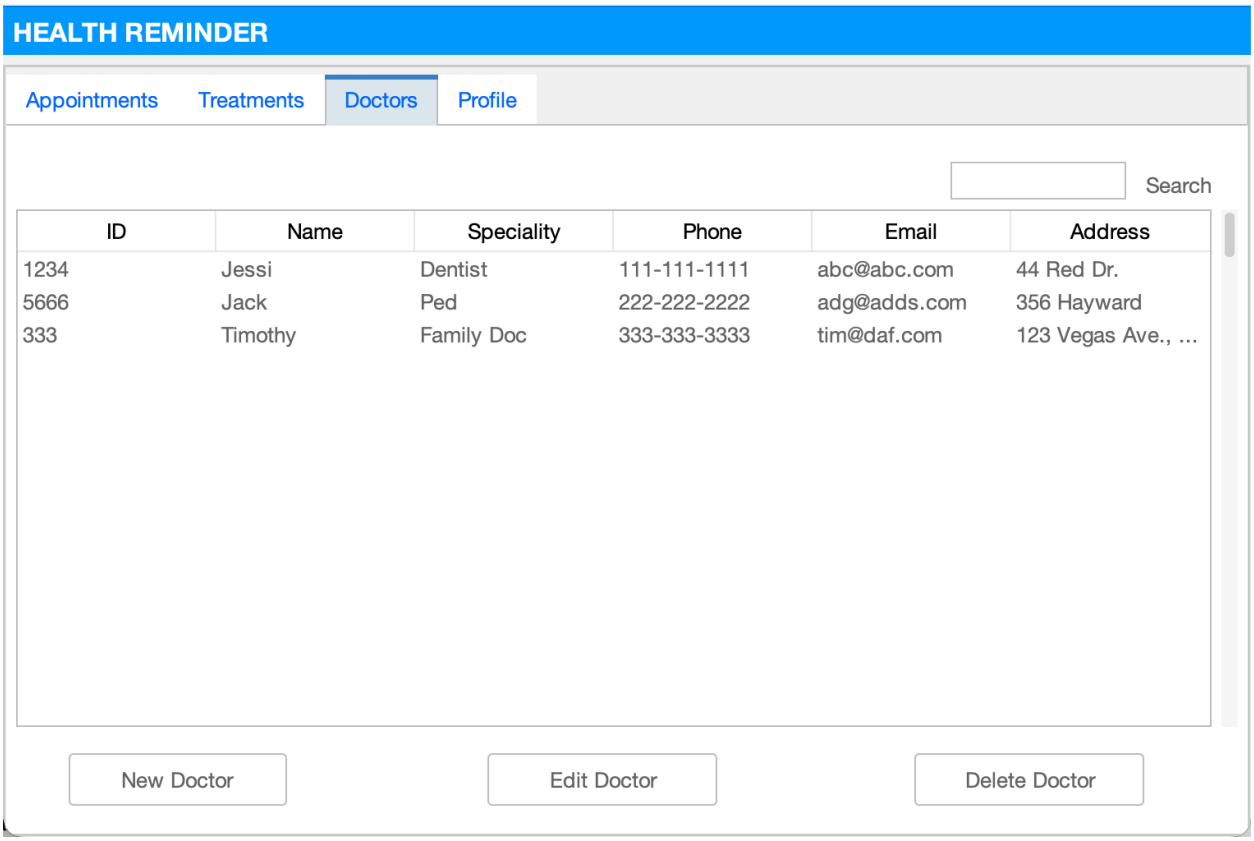
UML




User
AddAppointmentController
AuthService
AppointmentStore
AppointmentController
enterAppointmentDetails()
addAppointment(data)
validateInput()
saveAppointment(appointment)
success
success
showConfirmation()
getAppointments()
appointmentList
sortList()
displayList()
User
AddAppointmentController
AuthService
AppointmentStore
AppointmentController

Doctors

GUI

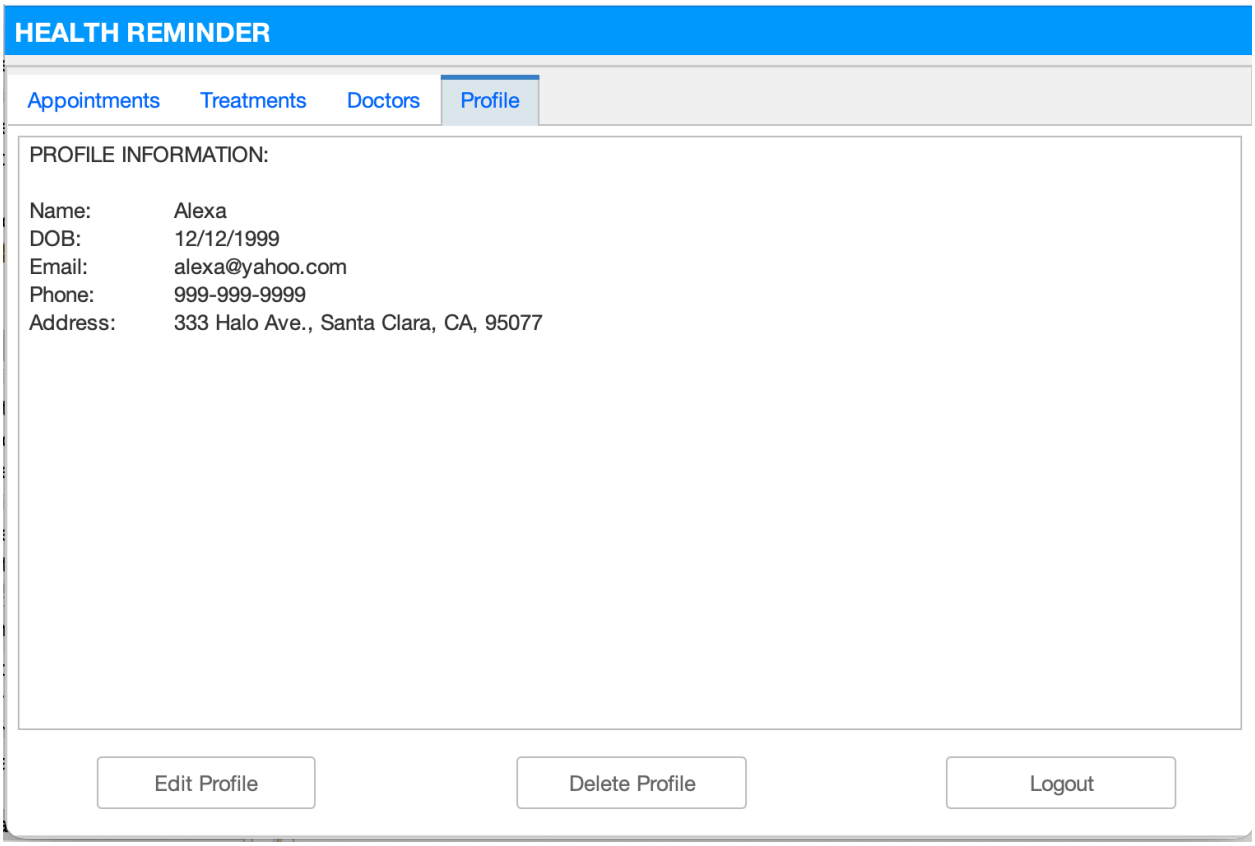

HEALTH REMINDER
Appointments
Treatments
Doctors
Profile
Search
ID
Name
Speciality
Phone
Email
Address
1234
Jessi
Dentist
111-111-1111
abc@abc.com
44 Red Dr.
5666
Jack
Ped
222-222-2222
adg@adds.com
356 Hayward
333
Timothy
Family Doc
333-333-3333
tim@daf.com
123 Vegas Ave., ...
New Doctor
Edit Doctor
Delete Doctor

UML

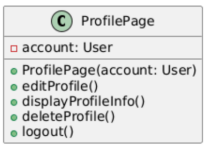
DoctorManager
doctorList: List<Doctor>
addDoctor(doctor: Doctor): void
editDoctor(id: int, field: String, value: String): void
removeDoctor(id: int): void
getDoctor(id: int): Doctor
listDoctors(): List<Doctor>
Doctor
doctorID: int
name: String
specialty: String
phone: String
email: String
address: String
Doctor(doctorID: int, name: String, specialty: String, phone: String, email: String, address: String)
getDoctorId(): int
getName(): String
getSpecialty(): String
getPhone(): String
getEmail(): String
getAddress(): String
setName(name: String)
setSpecialty(specialty: String)
setPhone(phone: String)
setEmail(email: String)
setAddress(address: String)
toString(): String

Profile

GUI



UML



6. Architecture & Tech Stack

- **Language**: Java 21
- **GUI**: JavaFX 21 + SceneBuilder (FXML)
- **Persistence**: SQLite via JDBC (lightweight, no server required)
- **Email**: JavaMail API
- **Build Tool**: Maven
- **IDE**: IntelliJ IDEA (recommended)

7. Scope Boundaries

In Scope

- All functions listed in Section 2
- Hard-coded data
- Basic email sharing via desktop client or JavaMail

Out of Scope

- Cloud synchronization
- Multi-user / family accounts
- Mobile version
- Push notifications
- Advanced reporting / analytics

8. Risks & Assumptions

Risks

- Reminder accuracy on minimized application
- Data privacy (local storage only)
- Email delivery reliability

Assumptions

- Users have a desktop environment with Java 21 installed
- Single-user application
- Internet required only for email feature

9. Deliverables & Timeline

- Week 2: Complete UML diagrams for all modules
- Week 3: GUI prototypes (SceneBuilder FXML)
- Week 5: Core CRUD operations + database integration
- Week 7: Email functionality, testing, and final demo

Appendix A – Team & Responsibilities

Team Members

- Amy Schmieder
- Jincy Jacob
- Trong Khoi Vu
- Leandro Kirchheim Coutinho
- Tam Nguyen

Task Assignment

- Jincy Jacob: UML for login and register
- Amy Schmieder: UML for medication management
- Trong Khoi Vu: UML for Doctor management
- Leandro Kirchheim Coutinho: UML for profile page
- Tam Nguyen: UML for Calendar

Project Progress

- Finalized the project idea
- Discussed and defined the application's functions and workflow, including the flowchart diagram
- Currently developing UMLs for each function, then will move to implementation and GUI design

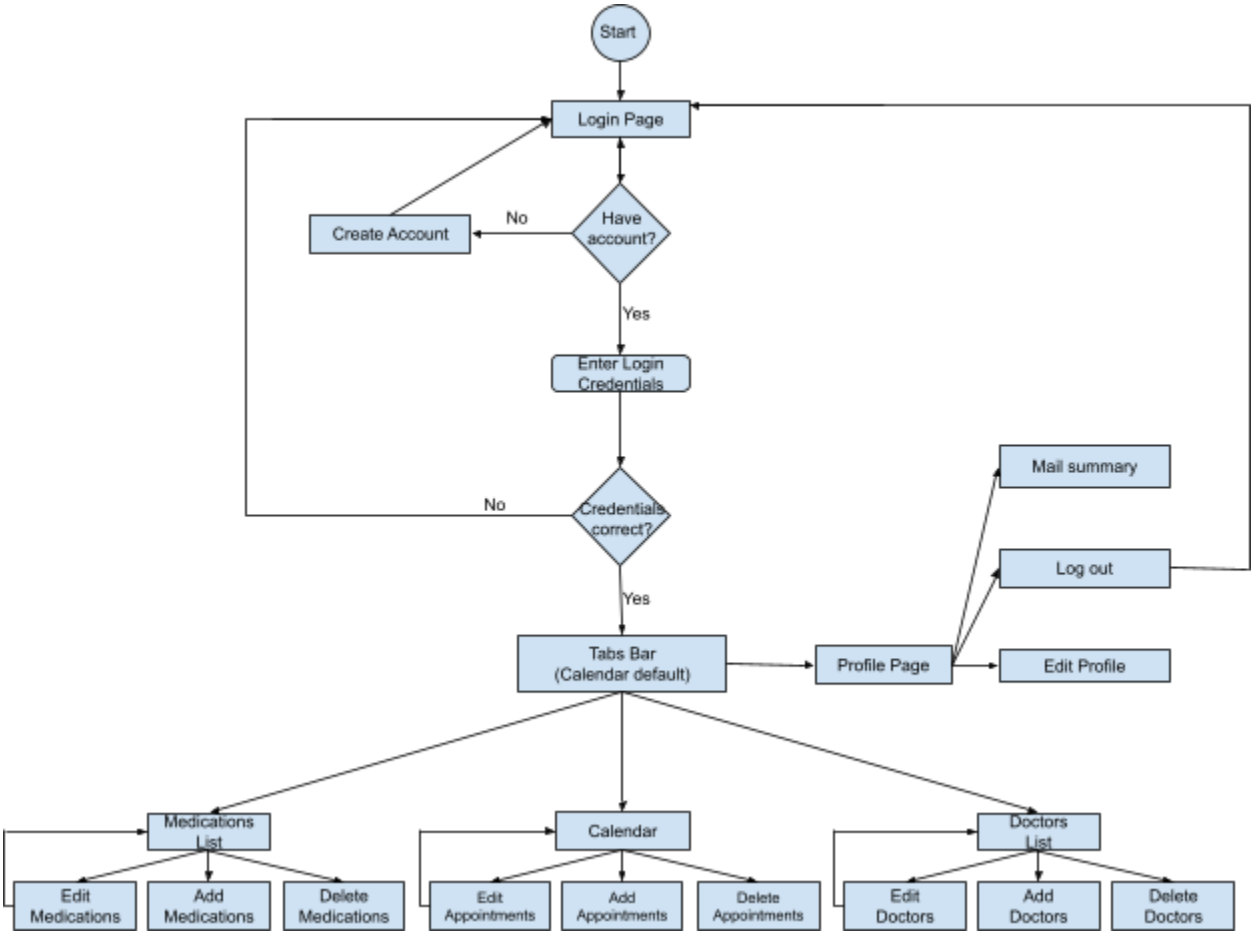



Figure 1: Proposed Use Case Flow Diagram